

**Name : P. Yaswanth**

**Reg No : 192524167**

### **Experiment: Minimax Algorithm for Gaming**

#### **Aim**

To implement the **Minimax Algorithm** using Python for decision-making in a two-player game.

#### **Objectives**

- To understand the concept of the Minimax algorithm.
- To implement the Minimax algorithm using Python.
- To determine the optimal move for a player in a game.
- To understand game tree evaluation in Artificial Intelligence.
- To analyze how Minimax helps in decision-making for two-player games.

#### **Algorithm**

1. Create a game tree with terminal node values.
2. Define the Minimax function with the current node, depth, and player type.
3. If the current node is a leaf node, return its value.
4. If it is the maximizing player's turn, select the maximum value among the child nodes.
5. If it is the minimizing player's turn, select the minimum value among the child nodes.
6. Continue recursively until the root node is evaluated.
7. Display the optimal value obtained by the maximizing player.

#### **Python Program**

# Minimax Algorithm

```
def minimax(depth, nodeIndex, isMax, scores, height):
```

```
# Base case: Leaf node reached

if depth == height:

    return scores[nodeIndex]


if isMax:

    return max(

        minimax(depth + 1, nodeIndex * 2, False, scores, height),

        minimax(depth + 1, nodeIndex * 2 + 1, False, scores, height)

    )
else:

    return min(

        minimax(depth + 1, nodeIndex * 2, True, scores, height),

        minimax(depth + 1, nodeIndex * 2 + 1, True, scores, height)

    )
```

# Driver Code

```
scores = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
height = 3
```

```
result = minimax(0, 0, True, scores, height)
```

```
print("The optimal value is:", result)
```

### **Sample Output**

The optimal value is: 12

## Out Put

```
# Minimax Algorithm

def minimax(depth, nodeIndex, isMax, scores, height):

    # Base case: Leaf node reached
    if depth == height:
        return scores[nodeIndex]

    if isMax:
        return max(
            minimax(depth + 1, nodeIndex * 2, False, scores, height),
            minimax(depth + 1, nodeIndex * 2 + 1, False, scores, height)
        )
    else:
        return min(
            minimax(depth + 1, nodeIndex * 2, True, scores, height),
            minimax(depth + 1, nodeIndex * 2 + 1, True, scores, height)
        )

# Driver Code
scores = [3, 5, 2, 9, 12, 5, 23, 23]
height = 3

result = minimax(0, 0, True, scores, height)

print("The optimal value is:", result)
```

The optimal value is: 12

=== Code Execution Successful ===

## Result

The Minimax algorithm was successfully implemented using Python. The program evaluated the game tree and determined the optimal value for the maximizing player.

## Conclusion

The experiment demonstrated the implementation of the **Minimax algorithm**, a fundamental decision-making algorithm in Artificial Intelligence. By recursively evaluating the game tree, the algorithm selected the optimal move while assuming that both players play optimally. Minimax is widely used in two-player games such as Tic-Tac-Toe, Chess, and Checkers to determine the best possible move.